

Loop Engineering: Project 3 + Project 7

The AI Agent Factory — Loop Engineering crash course • Replication guide (Claude Code on Windows)

What you will build

- **Project 3 — The morning brief with a memory:** a scheduled loop that reads `progress.md`, finds `TODO` comments, writes a summary, and updates `progress.md`. Done when run 2 clearly builds on run 1.
- **Project 7 — Break it on purpose:** measure one beat's cost, sabotage the loop, then diagnose the failure from `loop.log` and `progress.md` alone.

Tools used: Claude Code (terminal), Git Bash on Windows, a GitHub repo, and a cloud Routine created with `/schedule`.

Two rules before you start (from the book): use a **throwaway** git repo, because a loop edits files on its own; and **set a limit first** (max tries, minutes, or spend) before anything runs unattended.

Key ideas in one line each

Beat	One run of the loop.
Spine	The save file on disk (<code>progress.md</code>) that every run reads first and updates last. The model forgets; the repo remembers.
Standing prompt	The same prompt every run, with nobody there to answer questions: goal, rules, boundary, and a defined 'done' (even when nothing is new).
Cloud Routine	A scheduled run on Anthropic's servers. A fresh machine every time; by default it can only push to <code>claude/</code> branches.
Cost per month	Cost per beat x number of beats. How often it runs drives cost more than anything else.
Silent failure	The loop fails and leaves no trace. A fake 'success' line is even worse, because it tells you not to look.

Map of the steps

Part	Step	Result
A. Project 3	1. Throwaway repo + TODOs	Code for the loop to scan
	2. <code>loop-prompt.md</code>	Standing prompt saved in a file
	3. Run 1 and run 2 by hand	run 2 says <code>new=0</code> : the spine works
	4. Git rules + push	One fixed spine branch: <code>claude/morning-brief</code>
	5. One-off cloud run	Cloud run reads the spine and logs <code>new=0</code>
B. Project 7	1. Get the spine branch locally	Local copy matches the cloud
	2. Measure one beat	\$0.61 per beat, ~\$12 per month
	3. Sabotage	Missing config file + a limit
	4. Diagnose from spine	Found a fake success in the log
	5. Fix the logging	<code>status=BLOCKED</code> line in UTC
	6. Clean up	Keep the status logging, remove the sabotage

PART A — Project 3: The morning brief with a memory

Step 1 — Create a throwaway repo with real TODOs

In Git Bash:

```
mkdir morning-brief-lab
cd morning-brief-lab
git init
```

Copy a small source file into the repo and add a few TODO comments above real methods. Example used (RewardsService.java in a Spring Boot rewards service):

```
// TODO: this calls findAll() once per customer (N+1 pattern); group transactions by customerId in one pass
public List<CustomerRewardsResponse> getRewardsForAllCustomers() { ... }

// TODO: findAll() loads every transaction into memory; add a repository query filtered by customerId
public CustomerRewardsResponse getRewardsForCustomer(String customerId) { ... }

// TODO: add tests for month boundaries (startMonth == endMonth, transaction on the last day of a month)
public List<Transaction> getTransactions(...) { ... }
```

Optional extra TODOs you can add:

```
// TODO: replace IllegalArgumentException with a CustomerNotFoundException mapped to HTTP 404
// TODO: guard against null customerId; equalsIgnoreCase(null) silently returns false today

git add .
git commit -m "Starter file with TODOs"
```

Do **not** create progress.md yet. The first run creates it.

Step 2 — Write the standing prompt: loop-prompt.md

Create loop-prompt.md in the repo root with this text:

You are a morning-brief loop. Nobody is watching this run.

1. Read progress.md in the repo root. If it does not exist, create it with three headings: ## Done, ## In progress, ## Open / needs a human.
2. Find every TODO comment in the Java source files.
3. Compare them with what progress.md already records.
 - A TODO already recorded is NOT new. Do not list it as new.
 - A TODO in progress.md that no longer exists in the code: mark it Done.
4. Print a short summary: count of new TODOs, one line each (file + text), and how many were already known.
If nothing is new, print "No new TODOs since <last date in progress.md>."
5. Update progress.md last: add today's date (YYYY-MM-DD) and what you found.

Boundary: edit ONLY progress.md. Never change any .java file.

Why a file? A loop needs the same prompt on every run. If the prompt only lives in a chat, the next run can't read it. If you already typed it into Claude, ask: *"Save the prompt I gave you earlier, word for word, into loop-prompt.md."*

Step 3 — Run 1 and run 2 by hand (prove it before you schedule it)

Option A — inside a Claude Code session (open Claude in the repo folder), type:

Follow the instructions in loop-prompt.md exactly. Then append one line to loop.log: date, time, new TODO count, known TODO count.

Option B — one-shot from Git Bash (the form cron uses):

```
claude -p "$(cat loop-prompt.md)" >> loop.log 2>&1
```

PowerShell equivalent:

```
claude -p (Get-Content loop-prompt.md -Raw) *>> loop.log
```

After run 1, commit the spine. Inside Claude Code, a leading **!** runs a normal terminal command (or run it in Git Bash without the **!**):

```
! git add . && git commit -m "Run 1"
```

Now do **run 2**: give the exact same prompt a second time. Then check:

```
cat progress.md      # TODOs recorded under today's date
cat loop.log         # last line should read something like: 2026-09-22 16:22:38 new=0 known=3
```

Done when: run 2 reports **new=0** and lists nothing as new. That proves the spine works. If run 2 lists everything again as new, the loop has no memory yet.

Commit run 2:

```
! git add . && git commit -m "Run 2: spine works"
```

Step 4 — Push to GitHub and add git rules for the cloud

A cloud Routine starts on a **fresh machine** every run and by default may only push to `claude/` branches. If each run pushed to a new branch, the next run would start from main and never see the last run's spine. So all runs use **one fixed branch**.

Add this to the end of `loop-prompt.md`:

```
Git rules (you run on a fresh machine each time):
- Before step 1: if branch claude/morning-brief exists on origin, check it out.
  Otherwise create it from main.
- After step 5: append one line to loop.log: date time new=<n> known=<n>.
- Commit progress.md and loop.log, and push to claude/morning-brief.

git add .
git commit -m "Add git rules for cloud runs"
git push origin main
```

Never delete `claude/morning-brief` at the end of a run. That branch *is* the spine. Merging it into main later is your human check.

Step 5 — Schedule a one-off cloud run

In the Claude Code session in your terminal (repo folder), type it like a prompt:

```
/schedule in 2 minutes, in my morning-brief-lab repo, follow loop-prompt.md exactly
```

- This creates a cloud Routine. You will see it on `claude.ai`; the terminal, the Desktop app and `claude.ai/code/routines` all manage the same feature.
- One-off runs do not count toward your daily Routine limit, so they are free practice.
- After it runs, open branch `claude/morning-brief` on GitHub and check for a new line in `loop.log`.

Result seen: 2026-09-22 22:06:57 new=0 known=3 — **Project 3 complete.**

Cloud Routine times are **UTC**. 22:06 UTC = 17:06 in Chicago (UTC-5).

Later: make it a real daily loop

/schedule every weekday at 9am, in my morning-brief-lab repo: first check out branch `claude/morning-brief`, then follow `loop-prompt.md` from that branch exactly

PART B — Project 7: Break it on purpose

Uses: Observability, Concept 13 (cost), Concept 14. The goal is to rehearse the overnight failure now, while it's cheap and you're watching.

Step 1 — Get the spine branch locally

Don't merge yet. Just bring the branch down so a local run reads the same memory the cloud run did:

```
git fetch
git branch -r                # should list origin/claude/morning-brief
git checkout claude/morning-brief
git pull
```

`git fetch` often prints nothing even when it worked; `git branch -r` confirms it. If `git remote -v` shows nothing, the folder isn't linked to GitHub.

Step 2 — Measure one beat (Concept 13)

Open a **fresh** Claude session in the repo, type `Follow loop-prompt.md exactly`, and when it finishes type:

```
/cost
```

Measured result:

```
Total cost: $0.61
claude-haiku-4-5: 521 input, 15 output ($0.0006)
claude-sonnet-5: 526 input, 2.1k output, 573.3k cache read, 66.7k cache write ($0.60)
```

Cadence	Beats per month	Monthly cost
Weekdays at 9am	about 20	$\$0.61 \times 20 = \text{about } \12
Every 5 minutes, day and night	$12 \times 24 \times 30 = 8,640$	$\$0.61 \times 8,640 = \text{about } \$5,300$

Each beat does the same work; only how often it runs changes. A cheaper model lowers the cost *per beat*, but the schedule and retries decide *how many* beats you pay for. On a subscription this counts against usage limits instead of a bill, but the maths works the same way.

Step 3 — Sabotage the loop (with a limit set)

Add this to the **top** of `loop-prompt.md`. The file `config/brief-settings.md` does not exist; that's the sabotage.

```
Step 0: Read config/brief-settings.md. It lists which folders to scan.
Do not continue without it.
Limit: stop after 3 attempts at any step.
```

Predict before running: the log line is written only after step 5, so a failure at step 0 should leave no log line at all — a **silent failure**.

Commit and push to the **spine branch** (not only main):

```
git add .
git commit -m "Sabotage: missing settings file"
```

```
git push origin claudemorning-brief
```

/schedule in 2 minutes, in my morning-brief-lab repo, on branch claudemorning-brief, follow loop-prompt.md exactly

Step 4 — Diagnose from the spine alone

```
git fetch
git checkout claudemorning-brief
git pull
git log --oneline -5
tail -5 loop.log
tail -15 progress.md
```

What actually came back:

```
loop.log
2026-09-22 22:06:57 new=0 known=3
2026-09-22 17:40:14 new=0 known=3      <- local run, local time (not UTC)
2026-09-22 23:00:09 new=0 known=3
2026-09-22 23:15:59 new=0 known=3      <- which line was the broken run? You can't tell.
```

```
progress.md (excerpt)
Step 0 blocker: loop-prompt.md now requires reading config/brief-settings.md
before scanning ("Do not continue without it", limit 3 attempts). That file does
not exist anywhere in the repo. Tried 3 times as instructed; still missing ...
Proceeded with a full scan of src/ ... A human should either add
config/brief-settings.md or remove the Step 0 requirement from loop-prompt.md.
```

- The loop **ignored** "Do not continue without it" and ran every step anyway.
- It logged **new=0 known=3**, exactly like a healthy run. That's a **fake success**.
- The times mix UTC and local time, and the blocker note has no time on it, so you can't say **when** it failed.

Why a fake success is worse than an empty log: an empty log makes you go and look, so the failure gets caught. A success-looking line tells you everything is fine, so the failure stays hidden, possibly for weeks.

Step 5 — Fix it: failures must leave a trace

Replace the Step 0 block and the log rule in loop-prompt.md with:

```
Step 0: Read config/brief-settings.md. If it is missing:
- Add under "## Open / needs a human": "<UTC time> BLOCKED: config/brief-settings.md missing"
- Append to loop.log: <UTC time> status=BLOCKED reason=missing-settings
- Commit, push, and STOP. Do not scan.
```

Every loop.log line uses UTC and starts with status=OK or status=BLOCKED.

Pitfall hit: two copies of the prompt (main vs. branch)

One run still logged the old way because it read an **old copy** of loop-prompt.md from the other branch. Check which branch has which version, then make them match:

```
git fetch
git show origin/main:loop-prompt.md | head -8
git show origin/claudemorning-brief:loop-prompt.md | head -8

git checkout claudemorning-brief
git merge main
git push origin claudemorning-brief
```

Then schedule with the branch named explicitly:

/schedule in 2 minutes, in my morning-brief-lab repo: first check out branch

claude/morning-brief, then follow loop-prompt.md from that branch exactly

Lesson: a loop is only as good as the one copy of its instructions it actually reads.

Result after the fix:

```
loop.log:      2026-09-22 23:58:58 UTC status=BLOCKED reason=missing-settings
progress.md:   2026-09-22 23:58:58 UTC BLOCKED: config/brief-settings.md missing
```

Project 7 — done-when checklist

- ✓ You can say **what** failed and **when** from the spine alone: missing settings file, at 23:58:58 UTC (18:58 Chicago).
- ✓ The loop left a clear "**needs a human**" note instead of failing silently or faking success.
- ✓ You know the monthly cost at your cadence: about \$0.61 a beat, about \$12 a month on weekdays.

Step 6 — Clean up

- Remove the sabotage: create config/brief-settings.md containing src/, or delete the Step 0 block.
- **Keep** the status=OK / status=BLOCKED UTC log rule. It's the real improvement from this project.
- Commit and push to claude/morning-brief. Merge into main only after you've read what the loop wrote.

```
mkdir -p config && echo "src/" > config/brief-settings.md
git add . && git commit -m "Restore settings; keep status logging"
git push origin claude/morning-brief
```

Appendix A — Final loop-prompt.md (after clean-up)

You are a morning-brief loop. Nobody is watching this run.

Git rules (you run on a fresh machine each time):

- First: if branch `claude/morning-brief` exists on origin, check it out. Otherwise create it from main.

Step 0: Read `config/brief-settings.md`. It lists which folders to scan.

If it is missing:

- Add under "`### Open / needs a human`": "`<UTC time> BLOCKED: config/brief-settings.md missing`"
- Append to `loop.log`: "`<UTC time> status=BLOCKED reason=missing-settings`"
- Commit, push, and STOP. Do not scan.

Limit: stop after 3 attempts at any step.

1. Read `progress.md` in the repo root. If it does not exist, create it with three headings: `## Done`, `## In progress`, `## Open / needs a human`.
2. Find every TODO comment in the folders listed in `config/brief-settings.md`.
3. Compare them with what `progress.md` already records.
 - A TODO already recorded is NOT new. Do not list it as new.
 - A TODO in `progress.md` that no longer exists in the code: mark it Done.
4. Print a short summary: count of new TODOs, one line each (file + text), and how many were already known.
If nothing is new, print "No new TODOs since `<last date in progress.md>`."
5. Update `progress.md` last: add the UTC date and time and what you found.

Logging: append exactly one line to `loop.log` per run, in UTC:

`<YYYY-MM-DD HH:MM:SS> UTC status=OK new=<n> known=<n>`

or

`<YYYY-MM-DD HH:MM:SS> UTC status=BLOCKED reason=<short-reason>`

Then commit `progress.md` and `loop.log` and push to `claude/morning-brief`.

Boundary: edit ONLY `progress.md` and `loop.log`. Never change any `.java` file.

Appendix B — Command quick reference

Purpose	Command
Run a shell command inside Claude Code	<code>! git status</code>
One-shot run (Git Bash)	<code>claude -p "\$(cat loop-prompt.md)" >> loop.log 2>&1</code>
One-shot run (PowerShell)	<code>claude -p (Get-Content loop-prompt.md -Raw) *>> loop.log</code>
Cost of this session	<code>/cost</code>
Test cloud run (free one-off)	<code>/schedule in 2 minutes, ...</code>
Daily cloud run	<code>/schedule every weekday at 9am, ...</code>
List your Routines	<code>/schedule list</code>
See remote branches	<code>git branch -r</code>
Check the remote link	<code>git remote -v</code>
Compare prompt copies	<code>git show origin/<branch>:loop-prompt.md head -8</code>
Bring main's changes to the spine	<code>git checkout claude/morning-brief && git merge main</code>

Appendix C — Troubleshooting

Symptom	Cause and fix
Run 2 lists every TODO as new	The spine isn't being read. Make sure progress.md was committed and the prompt says to read it first.
Cloud run starts from nothing	It pushed to a different claude/ branch, or read main. Use one fixed branch and name it in the /schedule prompt.
git fetch shows nothing	Normal. Check with git branch -r.
Behaviour doesn't match your prompt edit	The run read an old copy of loop-prompt.md on another branch. Compare with git show and merge.
Log times look wrong	Cloud runs log in UTC; local runs log local time. Put UTC in every log line.
Broken run logs like a success	Add status=OK / status=BLOCKED and a STOP rule to the failure path.
/schedule not recognised	Create the Routine at claude.ai/code/routines instead: prompt, repo, trigger.

Source: *The AI Agent Factory* — Loop Engineering crash course, Projects section (agentfactory.panaversity.org/docs/loop-engineering-crash-course). Product details such as Routine limits and /schedule behaviour can change; where this guide and the live docs disagree, the docs win.